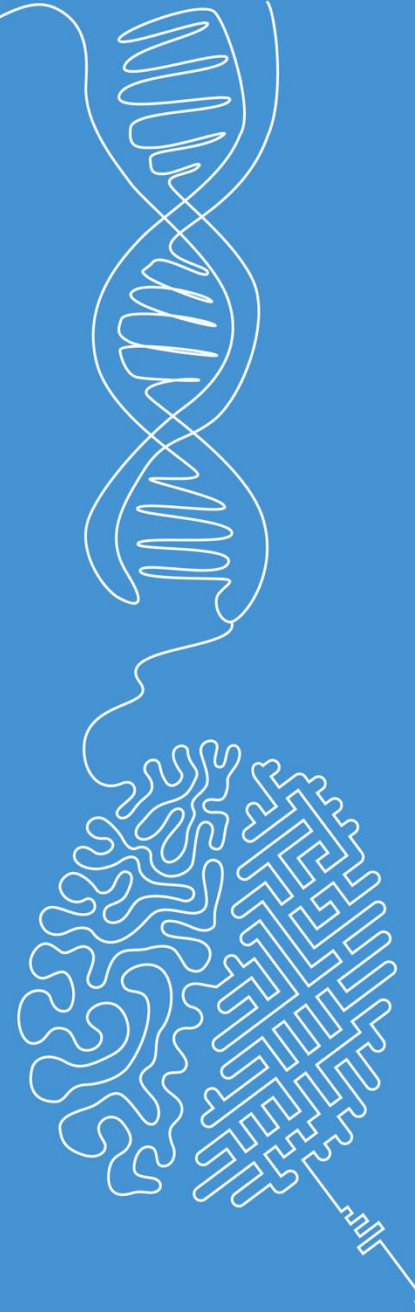


# Image compression

## Image and Signal Processing

Norman Juchler



# Compression

- Image file formats support compression to reduce the file size
- Lossless compression:
  - Reduces the file size without losing image quality
  - Use when you prioritize image quality over file size
- Lossy compression:
  - Reduces the file size by discarding some image data
  - Goal: discard “invisible” information or small details
  - Some methods to gradually set the compression level
  - Trade-off: Quality vs. file size

# Compression

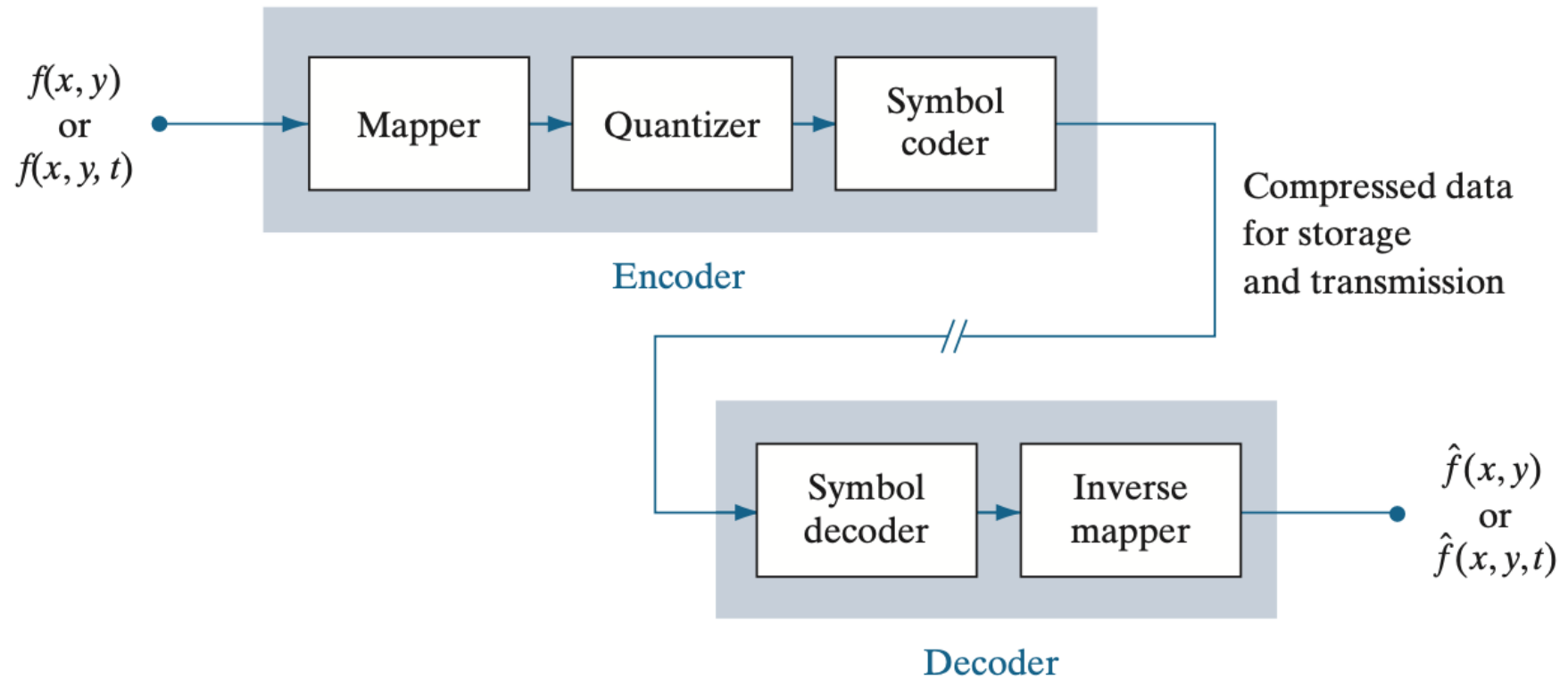


Original  
File size: 1.5 MB



High lossy compression  
File size: 92 KB

# Compression



Functional block diagram of a general image compression system.

(Applies to images  $f(x, y)$  and video frames  $f(x, y, t)$ ).

Source: Gonzales and Woods 2018

# Compression methods – lossless

## ■ Run Length Encoding (RLE)

- Encode consecutive data elements as a single value and count
- Example:
  - Input: `WWWWWWWWWWWWBWWWWWWWWWWB`
  - Output: `12W1B12W3B`
- Used in BMP and other bitmap formats

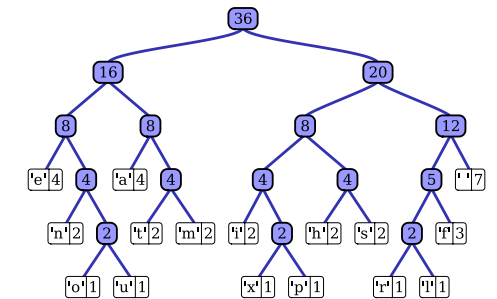
## ■ Huffman-Coding

- Is a dictionary-based compression algorithm: Assign variable-length codes to input data
- The coding is based on the frequency of occurrence of data elements
  - Frequent sequences are assigned short codes
  - Rarer sequences are assigned longer codes
- Used in PNG and JPEG files

**Example:** Huffmann coding.

**Idea:** Use a variable-length code for different tokens (or byte sequences), with the shortest codes used for the most frequent tokens.

**Input:** “this is an example of a huffman tree”. Tokens by frequency:



**Output:** Huffmann code table

| Char ↕ | Freq ↕ | Code ↕ | Char ↕ | Freq ↕ | Code ↕ |
|--------|--------|--------|--------|--------|--------|
| space  | 7      | 111    | s      | 2      | 1011   |
| a      | 4      | 010    | t      | 2      | 0110   |
| e      | 4      | 000    | l      | 1      | 11001  |
| f      | 3      | 1101   | o      | 1      | 00110  |
| h      | 2      | 1010   | p      | 1      | 10011  |
| i      | 2      | 1000   | r      | 1      | 11000  |
| m      | 2      | 0111   | u      | 1      | 00111  |
| n      | 2      | 0010   | x      | 1      | 10010  |

# Compression methods – lossless

## ■ Lempel-Ziv-Welch (LZW)

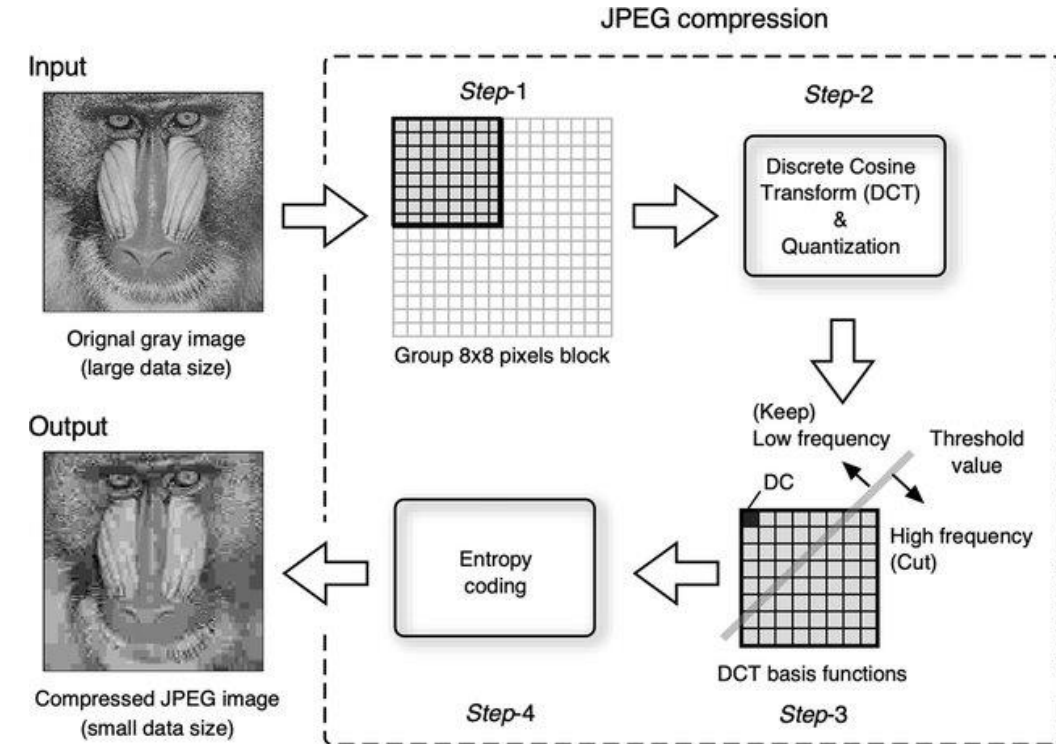
- A dictionary-based compression algorithm:
- Replace repeated data sequences with shorter codes
- Used in GIF, TIFF files

## ■ ZIP

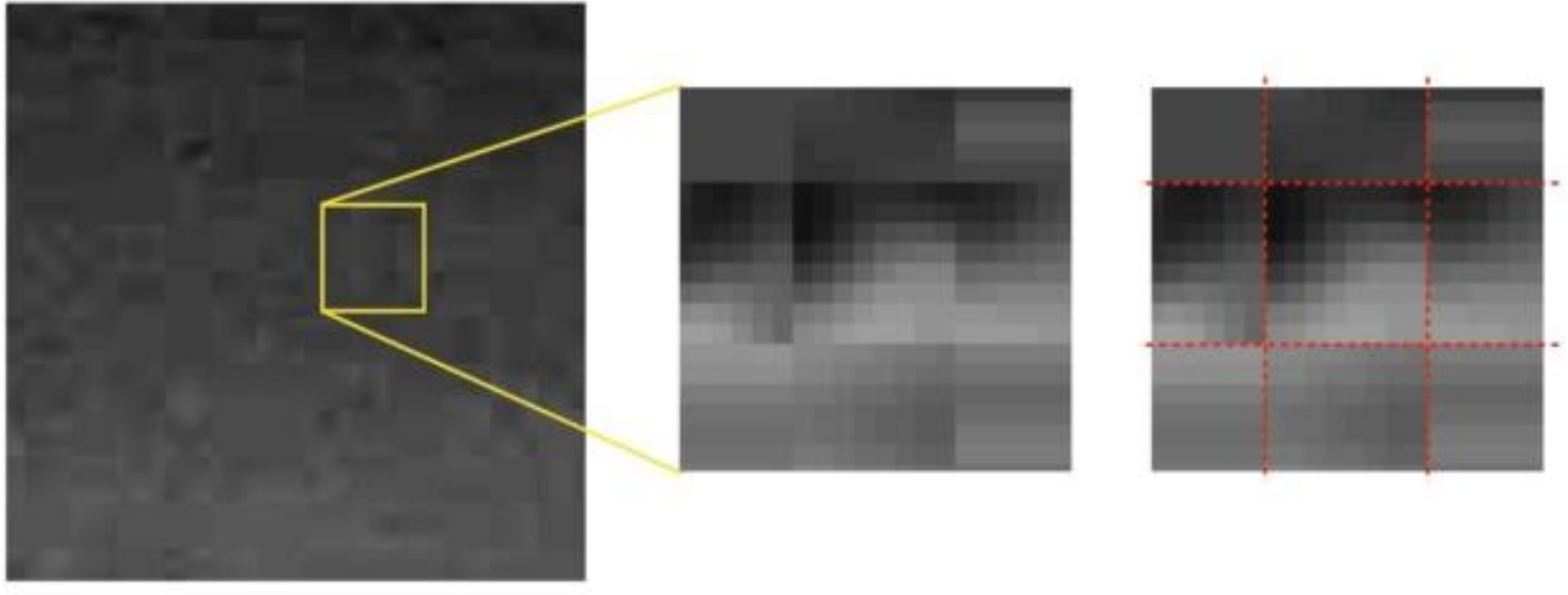
- Is related to LZW: Replace repeated data sequences with shorter codes
- Makes use of optimized compression algorithms, like [Deflate](#)
- Combined LZW and Huffman coding
- Used in TIFF files

# Compression methods – lossy JPEG

- Divide image into square blocks of pixels
- Use the Discrete Cosine Transform (DCT) to convert pixel values into frequency coefficients
- Quantize the frequency coefficients to reduce the amount of data (lossy)
- Use (lossless) Huffman coding to encode the quantized coefficients
- Allows for different levels of compression by adjusting the quantization tables
- Notes:
  - Other types of transforms (DFT, Wavelets) are possible, but less common



# Compression methods – lossy JPEG – compression artifacts



Common block artifacts from JPEG compression

# Compression methods – lossy JPEG – compression artifacts

